Individual Final Report

Jeongmin An

1. Introduction and Scope of My Contribution

This project investigated speculative decoding as a practical method for accelerating large language model inference under realistic GPU constraints. The group initially explored Qwen3-based model pairs, but because the early results were not stable enough for the final benchmark direction, the project shifted toward Gemma-based model pairs.

The final project developed two related acceleration directions:

1. **Standard speculative decoding with separate draft and target models**
   This was my main contribution area, centered on the Gemma-based benchmark module in Code/gemma-draft-pair/ and the Pair F/G comparison.
2. **EAGLE-3 draft-head training and inference**
   This was mainly handled by my teammate as a learned-speculation extension of the project.

My individual contribution focused on the standard Gemma draft-target speculative decoding track, especially the Pair F and Pair G comparison. After the project moved from early Qwen-based exploration to the final Gemma-based setup, I adapted the existing benchmark code and workflow so that the Gemma pairs could be loaded, executed, summarized, and compared consistently. I then ran the Pair F/G benchmark experiments across four task categories, multiple gamma values, and multiple temperature settings.

I organized the baseline and speculative decoding outputs into the F_final and G_final folders, preserved the final metrics in summary.csv files, and worked with the comparison figures under figures/FG_final. These outputs became the main evidence for my contribution and helped turn the Pair F/G comparison into a clear part of the group's final analysis. I also created the final presentation deck and translated the benchmark outputs into a clear presentation flow.

2. Background: Standard Speculative Decoding

Standard speculative decoding accelerates autoregressive generation by using two models with different roles. A smaller draft model first proposes several candidate tokens, and a larger target model verifies whether those tokens should be accepted. If multiple drafted tokens are accepted in one verification step, generation can move faster than calling the target model one token at a time.

This background was directly connected to my Pair F and Pair G experiments. In these experiments, gamma controlled how many tokens the draft model proposed before target verification. A larger gamma could improve speedup when many proposed tokens were accepted, but it could also increase overhead when many drafted tokens were rejected. Temperature was also important because higher temperature makes generation more random, which can reduce agreement between the draft and target models.

Based on this, I interpreted the benchmark results using multiple metrics together: speedup, acceptance rate, accepted length, TTFT, peak VRAM, and draft overhead. These metrics helped explain not only whether speculative decoding was faster, but also why Pair F and Pair G behaved differently across tasks.

3. My Experimental Responsibility

My benchmark work focused on comparing two Gemma-based draft-target configurations under the same experimental framework. Pair F used `google/gemma-3-12b-it` as the target model and `google/gemma-3-1b-it` as the draft model. Pair G used `google/gemma-4-31b-it` as the target model and the same google/gemma-3-1b-it as the draft model. This setup allowed me to compare whether a larger target model would actually provide better speculative decoding performance under the same benchmark conditions.

The experiments were run on an AWS g5.2xlarge instance with an NVIDIA A10G GPU and 24 GB of VRAM. This hardware setting was important because one of the goals of our project was to evaluate speculative decoding under a realistic mid-tier GPU environment rather than assuming access to larger A100 or H100 GPUs.

For each pair, I ran the benchmark across four task categories:

Task	Description
`humaneval`	Code generation
`triviaqa`	Question answering
`cnn_dailymail`	Long-context summarization
`writingprompts`	Open-ended creative generation

I also tested multiple speculation lengths and sampling temperatures:

Variable	Values
Gamma	1, 3, 5, 7, 10
Temperature	0.0, 0.6, 1.0
Prompts per task	50
Max new tokens	128

This benchmark design allowed the Pair F and Pair G results to be compared under the same task, gamma, and temperature conditions. It also made the later result organization easier because each experiment followed the same benchmark structure.

4. Output Organization and Evidence of My Work

After completing the benchmark runs, I organized the outputs into separate final folders for Pair F and Pair G so that the results could be reused for analysis, figures, the final report, and the group presentation. This was important because the benchmark produced many files across different tasks, gamma values, and temperature settings.

The main output locations were:

- Code/gemma-draft-pair/gemma_runs/outputs/F_final/
- Code/gemma-draft-pair/gemma_runs/outputs/G_final/

Each folder contained baseline outputs, speculative decoding outputs, and a final summary.csv file. The summary.csv files were important because they collected the benchmark metrics into a structured format for each pair.

I also used the summarized outputs to generate and review the final comparison figures saved under:

- Code/gemma-draft-pair/figures/FG_final/

These figures summarized speedup, acceptance behavior, temperature effects, TTFT, and VRAM usage. Together, the summary.csv files and FG_final figures became the main evidence for my contribution and helped connect the technical benchmark work to the final project analysis.

5. Results from Pair F

Pair F was the main standard speculative decoding configuration in my Gemma benchmark work. It used `google/gemma-3-12b-it` as the target model and `google/gemma-3-1b-it` as the draft model.

The best Pair F result for each task is summarized below.

Task	Best Gamma	Best Temperature	Mean TPS	Speedup	Acceptance Rate	Accepted Length	TTFT ms	Peak VRAM
HumanEval	10	0.0	14.22	1.476	0.9279	6.03	555.24	9.40
TriviaQA	5	0.0	12.47	1.295	0.8751	2.96	320.48	9.31
CNN/DailyMail	1	0.0	8.30	0.889	0.5342	0.53	525.01	9.77
WritingPrompts	1	0.0	8.43	0.871	0.4525	0.45	180.03	9.32

The Pair F results show that speculative decoding worked best on more structured tasks. HumanEval achieved the strongest speedup at 1.476x, and TriviaQA also improved over baseline with 1.295x speedup. These two tasks also had high acceptance rates, which means the draft model often proposed tokens that the target model accepted.

In contrast, CNN/DailyMail and WritingPrompts did not achieve speedup above baseline. Their best speedup values were below 1.0, which means the speculative decoding setup was slower than baseline for those tasks. This likely happened because long-context summarization and open-ended creative writing are less predictable, so the draft model's proposed tokens were accepted less often. Overall, Pair F was most effective when the task was structured and draft-target agreement was high. This made it a strong baseline for comparing against the larger Pair G setup.

6. Results from Pair G

Pair G was the larger-target comparison configuration in my Gemma benchmark work. It used `google/gemma-4-31b-it` as the target model and `google/gemma-3-1b-it` as the draft model. The purpose of Pair G was to test whether a larger target model would improve standard speculative decoding performance under the same benchmark setup.

The best Pair G result for each task is summarized below.

Task	Best Gamma	Best Temperature	Mean TPS	Speedup	Acceptance Rate	Accepted Length	TTFT ms	Peak VRAM

HumanEval	10	0.0	8.57	1.168	0.9021	5.15	907.95	19.41
TriviaQA	7	0.0	6.92	0.939	0.8090	3.14	632.32	19.32
CNN/DailyMail	5	0.0	5.23	0.762	0.7381	2.21	1567.04	20.44
WritingPrompts	3	0.0	4.40	0.597	0.5262	0.95	486.74	19.31

The Pair G results showed that a larger target model did not automatically improve speculative decoding performance. HumanEval was the only task where Pair G achieved speedup above baseline, reaching 1.168x. For TriviaQA, CNN/DailyMail, and WritingPrompts, the best speedup values remained below 1.0, meaning that speculative decoding was slower than baseline for those tasks.

Pair G also required much more GPU memory than Pair F. Its peak VRAM usage was around 19–20.5 GB, while Pair F stayed around 9–10 GB. Since the experiments were run on an A10G GPU with 24 GB of VRAM, Pair G technically fit within the hardware limit, but it left much less memory headroom. This made Pair G less practical for repeated benchmark sweeps under our available A10G hardware setting.

Overall, Pair G was useful as a comparison case because it showed that model size alone does not determine speculative decoding performance. Even with a larger target model, Pair G's higher memory cost and weaker speedup pattern made it less practical than Pair F in our benchmark environment.

7. Pair F versus Pair G Comparison

The direct Pair F/G comparison was the central result of my Gemma benchmark work. Since both pairs used the same draft model but different target models, this comparison helped show whether increasing the target size actually improved speculative decoding performance.

Comparison Point	Pair F	Pair G	Interpretation
Target model	Gemma-3-12b-it	Gemma-4-31b-it	Pair G used the larger target
Draft model	Gemma-3-1b-it	Gemma-3-1b-it	same draft model
Best HumanEval speedup	1.476x	1.168x	Pair F was faster
Best TriviaQA speedup	1.295x	0.939x	Pair F exceeded baseline
Best CNN/DailyMail speedup	0.889x	0.762x	Both were below baseline
Best WritingPrompts speedup	0.871x	0.597x	Pair F was closer to baseline
Peak VRAM range	About 9-10GB	About 19-20.5GB	Pair F used less memory

This comparison showed that Pair F was the stronger and more practical standard speculative decoding setup in our benchmark. Although Pair G used a larger target model, it did not produce better overall speedup and required almost twice as much GPU memory. This supported the main conclusion of my benchmark work: practical acceleration depends on the balance between speedup and hardware cost, not only on using a larger target model.

8. Figure-Based Evidence and Interpretation

In addition to the summary.csv files, the figures saved under figures/FG_final were important evidence for my benchmark work. They helped summarize the Pair F and Pair G results visually and supported the final presentation.

8.1 Figure 1. Pair F vs Pair G

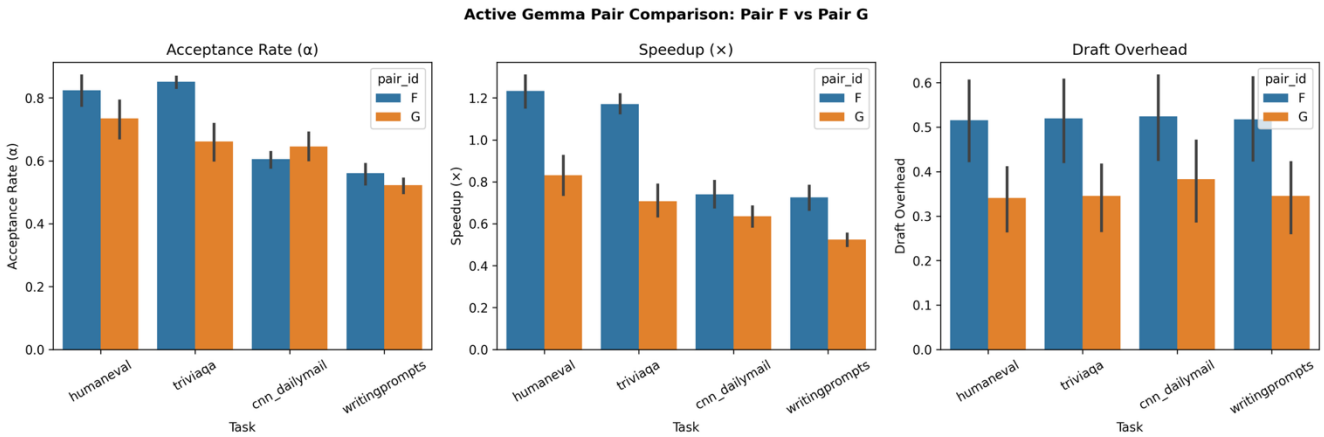

Figure 1 summarizes the direct Pair F and Pair G comparison across acceptance rate, speedup, and draft overhead. Pair F generally achieved stronger speedup, while the overhead plot shows why speedup should be interpreted together with computational cost.

8.2 Figure 2. Speedup vs Gamma

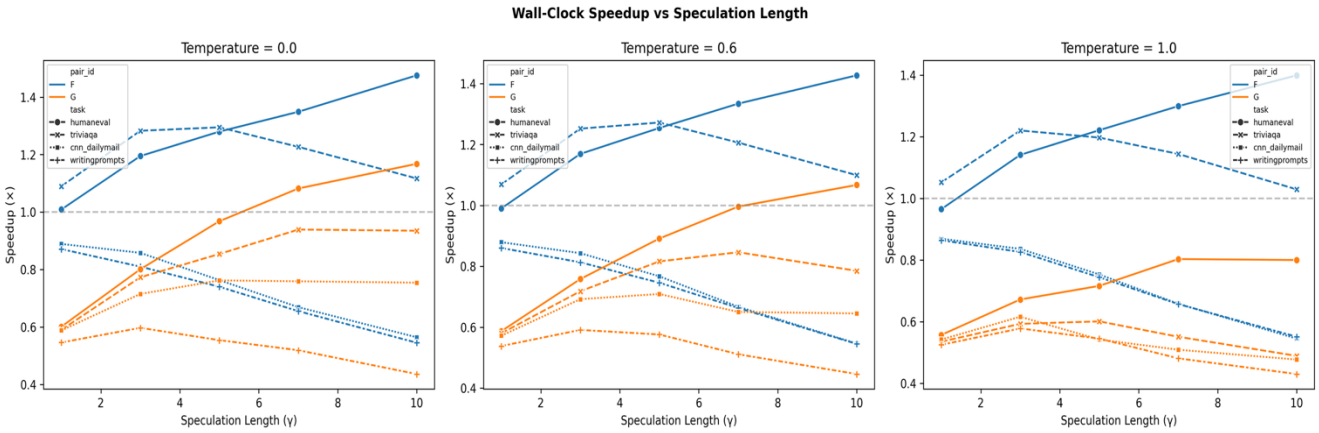

Figure 2 shows the relationship between gamma and speedup. Larger gamma did not always improve performance; it helped mainly when the draft model's proposed tokens were accepted often enough.

8.3 Figure 3. Acceptacne Rate Heatmap

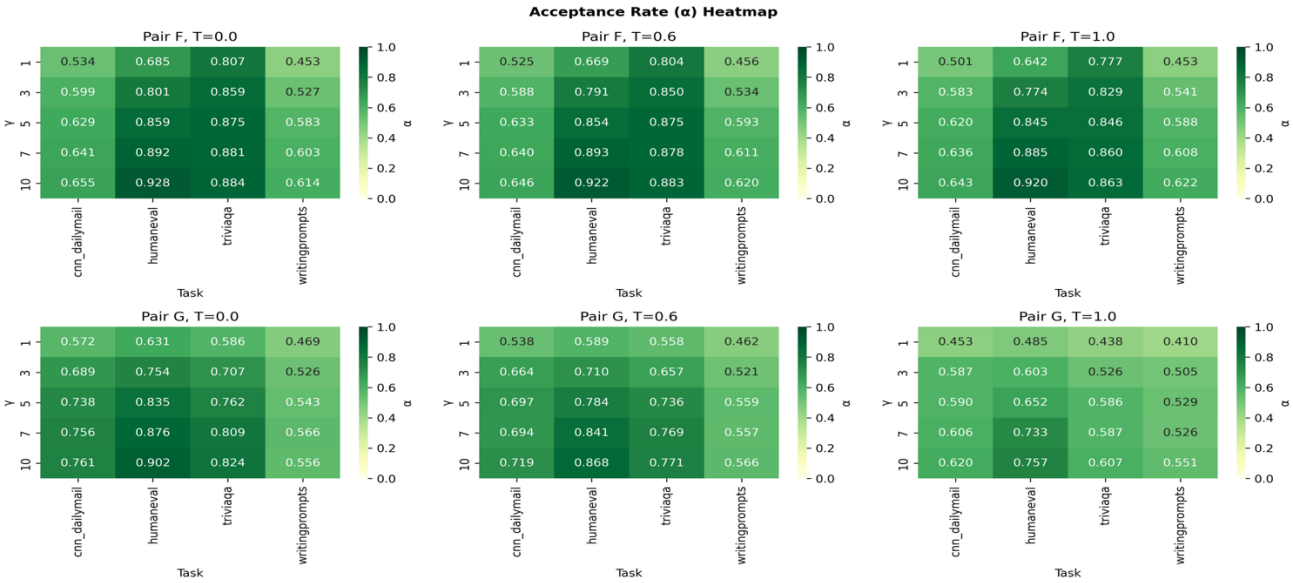

Figure 3 shows the acceptance-rate pattern across tasks and settings. Higher acceptance rates for tasks such as HumanEval and TriviaQA helped explain why these tasks showed stronger speculative decoding results, especially for Pair F.

8.4 Figure 4. VRAM Usage

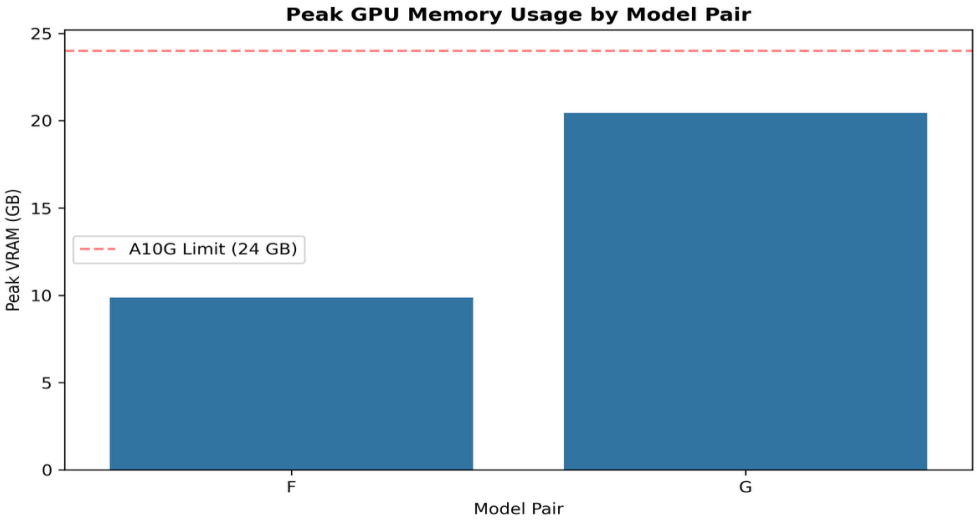

Figure 4 compares peak VRAM usage between Pair F and Pair G. Pair F used much less GPU memory and left more memory headroom on the 24 GB A10G GPU, making it the more practical setup for repeated benchmark runs.

9. What I Learned

This project helped me move from a classroom-level understanding of autoregressive LLM generation to a more practical understanding of inference acceleration. While I had learned that decoder-style language models generate text one token at a time, the Pair F and Pair G benchmarks showed me how that process affects throughput, latency, and GPU memory usage in a real experiment.

I also learned that a larger target model does not automatically lead to better speculative decoding performance. Pair G used a larger target model, but it required much more VRAM and did not consistently produce stronger speedup than Pair F. This helped me understand that practical acceleration depends on the balance between model size, draft-target alignment, acceptance behavior, and hardware cost.

Another important lesson was that benchmark results should be interpreted beyond a single metric. Speedup was important, but it was not enough by itself. I had to consider acceptance rate, accepted length, TTFT, peak VRAM, and draft overhead together to explain why Pair F and Pair G behaved differently across tasks.

10. Summary and Conclusion

My individual contribution focused on the standard Gemma speculative decoding benchmark, especially the Pair F and Pair G comparison. After the project moved from early Qwen-based exploration to the final Gemma-based setup, I adapted the existing benchmark code and workflow for the Gemma experiments, ran the Pair F/G benchmark runs, organized the outputs, and summarized the results through summary.csv files and comparison figures. The main artifacts from my work were the completed F_final and G_final output folders, the structured summary.csv files, and the comparison figures saved under figures/FG_final.

The benchmark results showed that Pair F was more practical than Pair G under our hardware setting. Pair F achieved stronger speedup on structured tasks such as HumanEval and TriviaQA while using much less GPU memory. Pair G was useful as a comparison case because it showed that using a larger target model does not automatically produce better speculative decoding performance.

Overall, my work contributed to both the technical execution and the final communication of the standard speculative decoding part of the project. In addition to completing and organizing the Pair F/G benchmark results, I created the final presentation deck for the group and structured the benchmark findings into a clear flow so that the project motivation, method, experiments, and findings could be communicated effectively.

11. External/AI-Assisted Code Percentage

My individual coding contribution focused on adapting, debugging and using the Gemma-specific benchmark workflow for the Pair F and Pair G experiments. My work contributed to the shared speculative decoding benchmark system, but the full codebase was also supported by collaborative development and AI-assisted coding during implementation, debugging, and experiment organization.

For this report, I calculate the code contribution percentage based on the active Gemma benchmark script base under Code/core/, rather than the entire project repository. The active script base includes the executable scripts needed to run the benchmark pipeline.

Based on the measured active script base of 3,019 lines, I assign my direct contribution credit as approximately 25%. This represents my Gemma-specific adaptation, benchmark execution, output organization, debugging support, result interpretation, and figure generation. The remaining 75% is attributed to collaborative, external, or AI-assisted code/workflow support. This percentage is an attribution estimate for course reporting purposes, not a line-by-line forensic reconstruction from version history.

Category	Estimated Percentage
My direct contribution through Gemma-specific adaptation, benchmark execution, debugging support, output organization, and result interpretation	25%
Collaborative, external, or AI-assisted code/workflow support	75%

Total active script lines = 3019

Assigned individual contribution ≈ 755 lines

Assigned collaborative/external/AI-assisted portion ≈ 2264 lines

755 / 3019 × 100 ≈ 25.0%